

Conditional Access Coverage Matrix Build and Operations Guide

PowerShell and Microsoft Graph read only analyser with JSON output

Script	Invoke-ConditionalAccessCoverageMatrix.ps1
Version	1.0.0
Audience	Microsoft 365 administrators, security engineers and PowerShell learners
Purpose	Explain installation, operation, JSON output, analysis logic and source code structure

Outcome

This tool turns tenant Conditional Access configuration into an explainable scenario matrix. It records what each policy targets, what it excludes, which access and session controls it applies, and where the configuration may contain gaps or conflicting intent. It does not change the tenant.

Contents

- 1 Purpose and design boundaries
- 2 Architecture and data flow
- 3 Requirements and permissions
- 4 Quick start
- 5 Parameters
- 6 JSON report structure
- 7 Analysis rules and interpretation
- 8 Emergency access protection
- 9 Source code walkthrough
- 10 Error handling and troubleshooting
- 11 Validation and safe testing
- 12 Maintenance and extension points
- 13 Operational checklist
- 14 Reference material

How to use this guide

Start with the quick start if you only need to run the tool. Read the JSON and analysis sections before you treat a finding as actionable. Use the source code walkthrough to learn how the PowerShell functions cooperate. Use the troubleshooting section when authentication, Graph permissions, throttling or file output fails.

1 Purpose and design boundaries

Conditional Access policies are easy to create one at a time and hard to understand as a complete system. A policy name may describe its intent, but the effective scope depends on users, groups, applications, platforms, locations, risks, client application types, filters, grant controls, session controls and exclusions. The script converts those separate objects into a consistent policy matrix and then runs conservative configuration checks.

What the tool does

- Reads Conditional Access policies from Microsoft Graph v1.0.
- Reads named locations and replaces location identifiers with readable names in the matrix.
- Reads built-in and custom authentication strength policies when the signed-in context permits it.
- Normalises every policy into one JSON scenario row.
- Detects defined baseline gaps, identical normalised scopes, large exclusion surfaces, selected contradictory patterns and emergency-access exposure.
- Writes a valid JSON report even when the main analysis fails after the output path is available.
- Adds timestamped diagnostics so an operator can trace warnings and partial results.

What the tool does not do

- It does not create, update, enable, disable or delete any Microsoft Entra object.
- It does not use sign-in logs or the Conditional Access What If engine.
- It does not expand nested group membership.
- It does not prove that an identified gap is exploitable. It reports that a known policy pattern was not detected.
- It does not calculate every possible intersection between partial scopes. Exact-scope overlap detection is intentionally simple and explainable.
- It does not replace change control, report-only deployment, pilot testing or administrator review.

Important. Conditional Access is cumulative. Multiple enabled policies can apply to one sign-in. This report shows the configuration and highlights likely design issues, but the final decision can depend on identity state, group membership, device state, location, authentication context and live service behavior.

Read only assurance

The implementation uses Invoke-MgGraphRequest with the GET method only. Its Graph paths are listed below. The separate authentication command obtains a token but does not modify tenant policy data.

Purpose	Method	Microsoft Graph v1.0 path
Conditional Access policies	GET	/identity/conditionalAccess/policies
Named locations	GET	/identity/conditionalAccess/namedLocations
Authentication strengths	GET	/policies/authenticationStrengthPolicies

2 Architecture and data flow

Conditional Access Coverage Matrix data flow

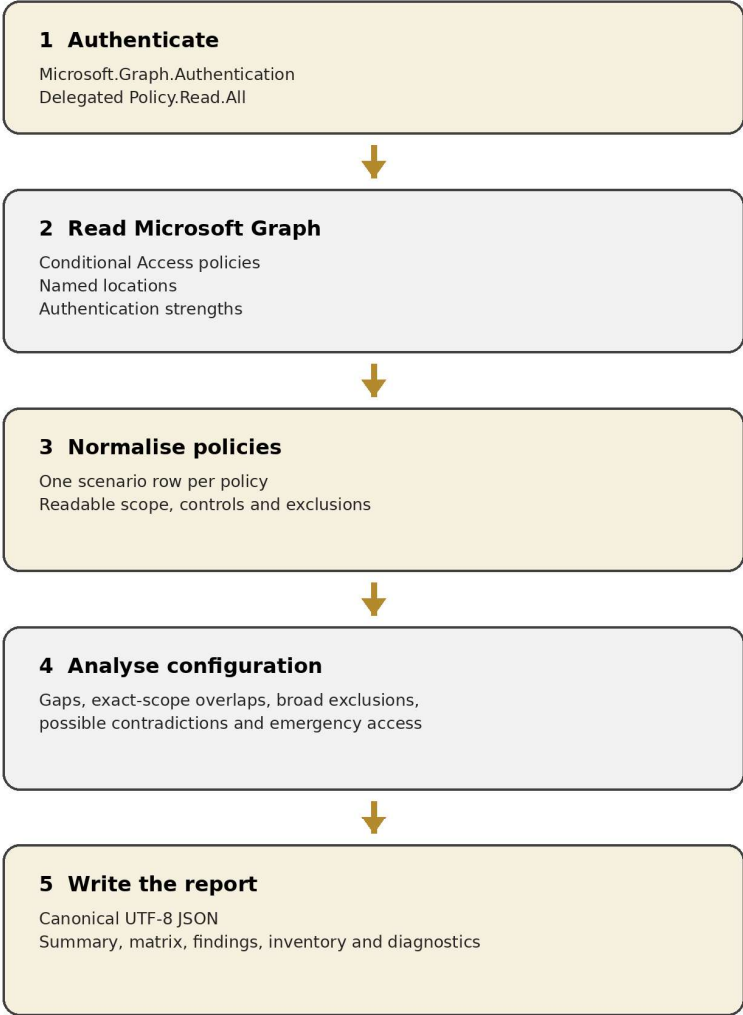


Figure 1 Data flow from delegated authentication to the JSON report

The design has five stages. Authentication creates a process-scoped Microsoft Graph session. The collection stage follows pagination and retries transient failures. The normalisation stage converts Graph objects into stable rows. The analysis stage reads those rows rather than the original policies. The output stage serialises one report object to UTF-8 JSON.

Why normalisation is separate from analysis

Separating the two operations keeps the rules understandable. `ConvertTo-ScenarioRow` handles Graph object shapes and null values. `Get-CoverageFindings` works with a predictable schema. If Microsoft adds a field, you

can update the normaliser without rewriting every finding rule. If you add a rule, you can usually work only with the matrix.

3 Requirements and permissions

Requirement	Minimum or recommended value	Reason
PowerShell	5.1 minimum; PowerShell 7 recommended	The script uses broadly compatible syntax. Microsoft recommends PowerShell 7 for the Graph SDK.
Module	Microsoft.Graph.Authentication	Provides Connect-MgGraph, Invoke-MgGraphRequest, Get-MgContext and Disconnect-MgGraph.
Delegated scope	Policy.Read.All	Reads Conditional Access policies and named locations. It is also a higher permission accepted by the authentication-strength endpoint.
Entra role	A supported read role such as Security Reader, Global Reader or Conditional Access Administrator	The signed-in account also needs a directory role that permits the operation.
Network	HTTPS access to Microsoft Graph and the sign-in endpoints	Required for authentication and API calls.

Install the small required module

```
Install-Module Microsoft.Graph.Authentication -Scope CurrentUser
Import-Module Microsoft.Graph.Authentication
```

The script intentionally uses only the authentication submodule and direct Graph requests. Installing the complete Microsoft.Graph package is not required.

Permission behavior

If there is no Graph context, the script connects with delegated Policy.Read.All and sets ContextScope to Process. The token cache is therefore limited to the running PowerShell process. If a context already exists, the script verifies that Policy.Read.All is present. It does not silently reconnect with broader permissions.

Least privilege. Policy.Read.All is the least-privileged delegated permission documented for listing Conditional Access policies and named locations. The authentication-strength list accepts Policy.Read.AuthenticationMethod as its least privilege permission and Policy.Read.All as a supported higher permission. The script uses one policy-read scope to keep the operator experience simple.

4 Quick start

First run

1. Copy Invoke-ConditionalAccessCoverageMatrix.ps1 to a controlled working folder.
2. Open PowerShell 7 or Windows PowerShell 5.1.
3. Install Microsoft.Graph.Authentication if it is not already installed.
4. Change to the script folder.
5. Run the audit with -Verbose.
6. Complete interactive Microsoft sign-in when prompted.
7. Open the timestamped JSON file shown in the console summary.

```
./Invoke-ConditionalAccessCoverageMatrix.ps1 -Verbose
```

Recommended emergency access run

```
./Invoke-ConditionalAccessCoverageMatrix.ps1 `
  -EmergencyAccessAccountObjectId '11111111-1111-1111-1111-111111111111', `
  '22222222-2222-2222-2222-222222222222' `
  -EmergencyAccessGroupObjectId '33333333-3333-3333-3333-333333333333' `
  -OutputPath 'C:\Reports\ConditionalAccess-Coverage.json' `
  -Verbose
```

Replace all example GUIDs. Use user object IDs, not user principal names, for the emergency-access accounts. Use the object ID of the dedicated emergency-access security group if the tenant uses one.

Device code authentication

```
./Invoke-ConditionalAccessCoverageMatrix.ps1 -UseDeviceCode -Verbose
```

Use device code when a browser cannot open on the system that runs the script. The command displays a code and a sign-in URL. Complete the sign-in from a suitable browser.

Return the report to the pipeline

```
$report = ./Invoke-ConditionalAccessCoverageMatrix.ps1 -PassThru
$report.Findings | Where-Object Severity -in 'Critical','High' |
  Format-Table FindingId, Category, Title -AutoSize
```

5 Parameters

Parameter	Type and default	Use
-----------	------------------	-----

Parameter	Type and default	Use
OutputPath	String; timestamped JSON in the current folder	Selects the JSON output file. Missing parent folders are created.
EmergencyAccessAccountObjectId	String array; empty	Enables direct account exclusion checks. Supply at least two object IDs.
EmergencyAccessGroupObjectId	String; empty	Checks whether enabled restrictive all-user policies exclude the dedicated group.
TenantId	String; empty	Passes a tenant ID or verified domain to Connect-MgGraph.
UseDeviceCode	Switch; false	Uses device-code authentication.
SkipAuthenticationStrengthInventory	Switch; false	Skips the separate authentication-strength list request. Embedded policy strength data remains in the matrix.
IncludeRawPolicies	Switch; false	Appends original policy objects to RawPolicies. Useful for troubleshooting but increases report size.
NoDisconnect	Switch; false	Keeps a connection created by the script. Existing user connections are never disconnected by default.
PassThru	Switch; false	Writes the report object to the pipeline in addition to the JSON file.
Verbose	Common parameter	Shows stages, API paths and progress information.

Input validation

Emergency-access identifiers are validated as 36-character GUID-shaped strings before the main code runs. OutputPath cannot be empty. MaxAttempts inside the Graph helper is constrained to 1 through 10. These checks reject common mistakes early, before authentication or data collection.

6 JSON report structure

Top-level property	Contents
SchemaVersion	Report schema version. Use this to protect downstream parsers from incompatible changes.
Tool	Name, script version, read-only flag and Graph API version.
Run	UTC times, tenant, account, PowerShell version, computer name and effective parameter choices.
Summary	Policy-state counts, inventory counts, total findings and severity counts.
ScenarioMatrix	One normalised record for every policy that could be parsed.

Top-level property	Contents
Findings	Ordered observations with category, severity, evidence, recommendation, policy IDs and confidence.
Inventory	Named locations and authentication-strength definitions.
EmergencyAccessAssessment	Identifiers supplied to the script and the group-expansion limitation.
Diagnostics	Timestamped information, warning and error entries.
AnalysisNotes	Interpretation boundaries that should travel with the report.
RawPolicies	Optional original Graph objects when IncludeRawPolicies is used.

Scenario matrix fields

Field group	Representative fields	Meaning
Identity	PolicyId, PolicyName, State, dates	Stable identity and enforcement state.
Subjects	Users, IncludeUsers, groups and roles	Readable summary plus raw identifier arrays.
Resources	Applications, UserActions, AuthenticationContexts	Cloud app and resource scope.
Conditions	Platforms, Locations, ClientAppTypes, risks, filters	Signals that must match before the policy applies.
Controls	AuthenticationStrength, GrantControls, SessionControls	Access requirements and session behavior.
Exclusions	Readable summary and raw exclusion arrays	Objects deliberately removed from scope.
Analysis key	ScenarioSignature	Stable text used to identify exact normalised scope overlap.

Example report fragment

```
{
  "SchemaVersion": "1.0",
  "Summary": {
    "PolicyCount": 18,
    "EnabledPolicyCount": 14,
    "FindingCount": 7,
    "FindingsBySeverity": {
      "Critical": 1,
      "High": 2,
      "Medium": 4,
      "Low": 0,
      "Information": 0
    }
  },
  "ScenarioMatrix": [ ... ],
  "Findings": [ ... ]
}
```

Failure JSON

If a fatal error occurs, the catch block attempts to write a smaller report with Status set to Failed, the exception message and all diagnostics gathered so far. This makes scheduled or remote runs easier to troubleshoot because a missing success report is not the only signal.

7 Analysis rules and interpretation

The rules are deliberately transparent. Each finding states what pattern was found or not found, which policies supplied the evidence and how confident the script is. Findings are sorted Critical, High, Medium, Low and Information, then assigned stable run-local IDs such as CA-001.

Category	Rule	Default severity	Interpretation
Gap	No enabled policies	Critical	The tenant returned no enforced Conditional Access policy rows.
Gap	No all-users and all-apps baseline	High	No single enabled policy contains both All tokens. Split coverage can still exist.
Gap	No MFA or authentication strength	High	No enabled policy uses the mfa grant or an authentication strength.
Gap	No legacy-authentication block	High	No enabled block targets Exchange ActiveSync or other legacy client types.

Category	Rule	Default severity	Interpretation
Gap	No administrator-role MFA	High	No enabled MFA policy contains directory role template IDs. Admins may still inherit all-user coverage.
BroadExclusion	Five or more direct exclusions	Medium	The policy has a comparatively large exception surface that deserves ownership review.
Overlap	Identical ScenarioSignature	Medium	Two or more enabled policies have the same normalised include conditions and filters.
Contradiction	Same signature has block and non-block grants	High	The included scenario appears both blocked and conditionally granted. Exclusions may still separate actual application.
Contradiction	All appears in an exclusion	High	The include and exclude design is likely ineffective or misleading.
Contradiction	Different persistent-browser modes on the same signature	Medium	Equivalent normalised scopes request different session persistence behavior.
EmergencyAccess	Insufficient identifiers or unexcluded restrictive scope	Medium to Critical	The tool cannot validate the design, or a supplied account may be restricted.

Why gap confidence is sometimes Medium

A static pattern check can miss equivalent designs. For example, an organisation can cover administrators through an all-users MFA policy instead of a role-targeted policy. It can block legacy authentication through several application-specific policies instead of one broad policy. The script therefore describes absence of the expected pattern rather than declaring the tenant insecure.

How exact-scope overlap works

ConvertTo-ScenarioRow constructs ScenarioSignature from sorted include-user, include-group, include-role, include-application, platform, location and client-app arrays. It also adds user risk, sign-in risk, device filter and application filter. Empty arrays become an asterisk. Case is normalised to lower case. Policies with identical signatures are grouped.

```
U=all;G=*;R=*;A=all;P=*;L=*;C=all;UR=*;SR=*;DF=*;AF=*
```

The signature deliberately excludes policy state, exclusions and controls. Those are the values that may explain why two policies with the same include scope are either a useful composition or a problematic duplication. The finding tells the reviewer to inspect them instead of silently collapsing the policies.

Block and grant interaction

When two enabled policies apply, controls are cumulative. A matching block policy prevents access even if another matching policy requests MFA or device compliance. The contradiction rule therefore reports exact normalised scopes that contain both block and non-block grant controls. It does not claim the policies are syntactically invalid; it signals that policy intent may be hard to understand or that exclusions are carrying hidden design weight.

8 Emergency access protection

Microsoft recommends at least two emergency-access accounts and advises excluding them from Conditional Access policies that block or restrict sign-in. The script treats this as a protected design requirement, but it requires explicit identifiers so it does not guess which accounts are emergency accounts.

Checks performed

1. If no account or group identifiers are supplied, create a Medium finding that validation is incomplete.
2. If fewer than two account IDs are supplied, create a High finding about redundancy.
3. For each supplied account, inspect every enabled policy with grant or session controls.
4. Treat the account as in scope when it is included directly or through All users.
5. Treat it as protected when the account is excluded directly or the supplied emergency-access group is excluded.
6. Create a Critical finding when the account appears to remain in a restrictive policy scope.
7. For the supplied group, create a High finding for each restrictive All users policy that does not exclude it.

Known emergency access limitation

The script does not call the group-membership API. It therefore cannot prove that a user belongs to the supplied group. It also cannot determine whether an account is included through another targeted group. The evidence text labels account findings as Medium confidence for this reason. Validate group membership and use the Conditional Access What If tool before changing policy.

Design guidance outside this script

- Use at least two cloud-only accounts with no federation dependency.
- Use phishing-resistant authentication that does not share the normal administrator authentication dependency.
- Keep permanent active Global Administrator assignments for the approved emergency accounts.
- Store credentials and devices securely and separately.
- Alert on every sign-in and audit event for these accounts.
- Test the full emergency process at least every 90 days and after material staffing or tenant changes.

9 Source code walkthrough

The script uses small functions with one clear responsibility. The main execution block calls them in a fixed sequence. This section explains the contract between the functions rather than repeating every line.

Function or block	Input	Output	Role in the pipeline
Add-Diagnostic	Level, stage, message, detail	Diagnostic object and optional console warning	Creates a common troubleshooting record.
Get-Array	Any value or null	Always an array	Prevents null and single-object edge cases.
Get-PropertyValue	Object, property name, default	Property value or default	Safely reads changing Graph object shapes under StrictMode.
Join-DisplayValue	Collection and empty label	Readable comma-separated text	Builds human-readable matrix columns.
Test-ContainsValue	Collection and target string	Boolean	Performs case-insensitive token checks.
ConvertTo-StableListText	Collection	Sorted lower-case signature text	Makes policy-scope comparison deterministic.
Invoke-GraphGetAll	URI, stage, retry limit	Complete object array	Handles GET, pagination and transient retries.
Resolve-LocationList	Location IDs and lookup map	Readable location array	Maps named-location IDs without discarding identifiers.
Format-GrantControls	Graph grantControls object	Readable control expression	Shows AND or OR composition, terms and strength.
Format-SessionControls	Graph sessionControls object	Readable session summary	Normalises sign-in frequency, persistence and related controls.
ConvertTo-ScenarioRow	One Graph policy and location map	One normalised row	Creates the canonical scenario matrix record.
Get-CoverageFindings	Matrix and emergency IDs	Ordered finding array	Runs all static analysis rules.
Get-SeverityCounts	Finding array	Count object	Builds the report summary.
Main try block	Parameters and Graph context	Report object and JSON	Coordinates collection, transformation, analysis and output.
Catch and finally	Exception and connection ownership	Failure JSON and safe disconnect	Preserves diagnostics and closes only a connection created by the script.

StrictMode and explicit error behavior

Set-StrictMode Version 2.0 catches common variable and property mistakes. ErrorActionPreference is Stop, so non-terminating errors enter try and catch blocks. Because Graph response shapes can contain missing optional properties, Get-PropertyValue is used throughout normalisation rather than direct property access for optional values.

Graph collection and retry loop

Invoke-GraphGetAll begins with the supplied URI. It calls Invoke-MgGraphRequest with GET and OutputType PSObject. If the response has a value property, each item is added to a typed generic list. If @odata.nextLink exists, the function repeats with that URL. Errors that look like throttling, a 5xx response, timeout or a temporary condition use exponential delays capped at 30 seconds. All other errors stop immediately with the stage name.

```
$response = Invoke-MgGraphRequest -Method GET -Uri $nextUri -OutputType PSObject

if ($response.PSObject.Properties['@odata.nextLink']) {
    $nextUri = [string]$response.'@odata.nextLink'
}
```

Normalisation details

ConvertTo-ScenarioRow first extracts conditions, users, applications, platforms, locations, devices, grants and sessions. It then converts every optional collection to an array. This prevents Count and enumeration failures when Graph returns null, a single object or an empty array. The function creates both readable text and raw arrays. Readable fields are convenient for humans; raw arrays support deterministic rule logic and downstream automation.

Main execution sequence

1. Confirm that Microsoft.Graph.Authentication is installed and import it.
2. Reuse a compatible Graph context or create a process-scoped delegated context.
3. Read policies and named locations. Attempt authentication strengths unless skipped.
4. Build a location ID lookup table.
5. Normalise every policy independently. Record and continue after a policy-specific normalisation failure.
6. Analyse the complete matrix and count severity levels.
7. Create concise location and authentication-strength inventories.
8. Build the ordered report object and optionally append raw policies.
9. Create a missing output directory, serialise to depth 30 and write UTF-8 without a byte-order mark.
10. Print a compact console summary and optionally return the object.

10 Error handling and troubleshooting

Symptom	Likely cause	Action
Module not installed	Microsoft.Graph.Authentication is absent in this PowerShell installation	Run Install-Module in the same PowerShell edition that will run the script.
403 or insufficient privileges	The token lacks Policy.Read.All, consent is missing, or the user lacks a supported Entra role	Check Get-MgContext, consent and role assignment. Use a reader role rather than a write role where possible.
Current context does not include Policy.Read.All	Another Graph session was opened with narrower scopes	Run Disconnect-MgGraph and start a fresh process, then run the tool again.
Authentication strength warning	Endpoint permission or tenant capability prevented the optional inventory read	Review the warning. Use SkipAuthenticationStrengthInventory if the separate list is not required.
Repeated 429 warnings	Graph throttling	Allow retries to complete. Reduce concurrent Graph tooling and run later if throttling persists.
Failure JSON only	A fatal error stopped the main pipeline	Read Error and Diagnostics. Re-run with -Verbose and IncludeRawPolicies only if collection succeeds.
Emergency-access finding seems wrong	Account is protected through group membership that the script cannot expand, or is included through another group	Check group membership and run the Entra Conditional Access What If evaluation.
Legacy-auth gap despite known blocking	The policy uses a different client type representation or coverage is split	Inspect ScenarioMatrix ClientAppTypesRaw and the original policies.
Output path access denied	The account cannot create the directory or file	Choose a user-writable path such as a folder under Documents.

Useful troubleshooting commands

```
Get-Module Microsoft.Graph.Authentication -ListAvailable
Get-MgContext | Format-List Account, TenantId, Scopes, AuthType

# Start a clean delegated session when the existing scopes are unsuitable.
Disconnect-MgGraph
Connect-MgGraph -Scopes 'Policy.Read.All' -ContextScope Process

# Inspect high-severity results after a pass-through run.
$report.Findings | Where-Object Severity -in 'Critical','High' |
    Select-Object FindingId, Category, Title, Confidence, PolicyIds
```

Diagnostics design

Each diagnostic contains TimestampUtc, Level, Stage, Message and Detail. Stage identifies the failing part, such as Authentication, ReadPolicies, ReadNamedLocations, ReadAuthenticationStrengths, NormalisePolicy or Fatal. Message is safe for a console summary. Detail retains the exception text for troubleshooting and can contain tenant-specific data; protect the JSON report accordingly.

11 Validation and safe testing

Pre-production validation

1. Run the script in a non-production tenant or with a Security Reader account first.
2. Confirm the JSON parses with ConvertFrom-Json.
3. Compare the Summary policy counts with the Entra admin center.
4. Select at least three policies: a broad policy, a policy with exclusions and a policy with session controls. Compare every matrix field with the portal.
5. Confirm that named locations show both the display name and identifier.
6. Supply test emergency-account and group IDs and validate the resulting findings manually.
7. Review every Critical and High finding. Record false-positive reasons before changing policy.
8. Use report-only mode and the What If tool for any remediation proposal.

JSON validation commands

```
$json = Get-Content '.\ConditionalAccess-Coverage.json' -Raw | ConvertFrom-Json
$json.SchemaVersion
$json.Summary
$json.ScenarioMatrix.Count
$json.Findings | Group-Object Severity
```

Static safety checks for the source file

```
Select-String -Path '.\Invoke-ConditionalAccessCoverageMatrix.ps1' `
-Pattern 'Invoke-MgGraphRequest'

Select-String -Path '.\Invoke-ConditionalAccessCoverageMatrix.ps1' `
-Pattern '-Method POST|-Method PATCH|-Method PUT|-Method DELETE'
```

The first command should find the GET request helper. The second should return no mutation methods. This is a simple source inspection, not a substitute for code signing or change control.

12 Maintenance and extension points

Add a new matrix field

1. Locate the related Graph object inside ConvertTo-ScenarioRow.
2. Read the optional property with Get-PropertyValue.
3. Convert collections with Get-Array.
4. Add a readable property and, if rules need it, a raw property.
5. Increase ConvertTo-Json depth only if the new field contains deeper nested objects.
6. Update SchemaVersion if downstream consumers could break.
7. Update the scenario matrix table in this guide and test null, empty, single and multiple values.

Add a new finding rule

Add rules to Get-CoverageFindings and create results only through New-Finding. Use the normalised matrix, not the raw policy, unless the required data cannot be represented cleanly. State evidence as an observation, state confidence explicitly and avoid claiming effective access when the script has not evaluated a real sign-in.

```
$findings.Add((New-Finding `
    -Category 'Gap' `
    -Severity Medium `
    -Title 'Expected pattern was not detected' `
    -Evidence 'Describe the exact matrix test.' `
    -Recommendation 'Describe a review action, not an automatic change.' `
    -Confidence Medium))
```

Automated execution

The script uses delegated interactive or device-code authentication. For unattended operation, use a dedicated app registration with certificate authentication and Policy.Read.All application permission. Protect the certificate, restrict execution, monitor the service principal and document ownership. Never place a client secret directly in the script.

Versioning guidance

- Increase the tool patch version for fixes that do not change JSON meaning.
- Increase the minor version when adding optional fields or finding rules.
- Increase SchemaVersion when renaming, removing or changing the meaning of existing JSON properties.
- Record the Graph API version in every report.
- Prefer Microsoft Graph v1.0. Use beta only in a separate experimental branch with clear warnings.

13 Operational checklist

Phase	Check
Before run	Confirm the target tenant, operator role, output folder and emergency-access object IDs.
Before run	Confirm that the script hash and version match the approved copy.
During run	Use -Verbose for first use and troubleshooting. Do not interrupt a temporary retry unless necessary.
After run	Confirm Status is not Failed and compare policy counts with the portal.
After run	Review Critical and High findings first, then Medium findings and all broad exclusions.
After run	Protect the JSON as tenant security configuration. Remove it when retention expires.
Before change	Validate group membership, exclusions and live behavior with What If and report-only mode.
Before change	Use peer review, change control, rollback planning and pilot groups.

14 Reference material

- [Microsoft Graph list Conditional Access policies](#)
- [Microsoft Graph list named locations](#)
- [Microsoft Graph list authentication strength policies](#)
- [Microsoft Graph Conditional Access policy resource](#)
- [Install the Microsoft Graph PowerShell SDK](#)
- [Microsoft Graph PowerShell authentication commands](#)
- [Manage emergency access administrator accounts](#)

Final interpretation rule

Treat the report as configuration evidence and a review queue. A finding should lead to validation, ownership and a controlled decision. It should not lead directly to an automatic policy change.